# Spatial Concept Distillation, step by step

What happens to one image, from the dataset to the trained student

Companion to the `concept-factors` report

**Abstract**

This document walks through SCD one step at a time. Every step has three parts. *In plain words* explains it so that a school student could follow. *Precisely* gives the shapes, formulas and numbers of the implementation. *Why this way* gives the reason for the choice. The running example is MetaShift (1,700 training images of cats and dogs, 130 concept factors). Numbers marked "illustrative" are made up to show a calculation; every other number comes from the code and the runs.

## Contents

**1 The whole method in one page** **2**

**2 Stage 1: preparation, once per dataset** **3**

2.1 Step 1.1: one vector per image from CLIP . . . 3

2.2 Step 1.2: SpLiCE turns the vector into a few words . . . 3

2.3 Step 1.3: group synonyms into concept factors . . . 4

2.4 Step 1.4: why plain CLIP is not enough for maps . . . 4

2.5 Step 1.5: reading the image through windows . . . 5

2.6 Step 1.6: from scores to a concept map . . . 5

**3 Stage 2: one training step** **6**

3.1 Step 2.1: two views of every image . . . 7

3.2 Step 2.2: the student looks at each view . . . 7

3.3 Step 2.3: cut the matching piece out of the concept map . . . 8

3.4 Step 2.4: the spatial concept loss . . . 8

3.5 Step 2.5: the SimCLR loss, unchanged . . . 10

3.6 Step 2.6: add the two losses and update . . . 11

**4 Stage 3: evaluation** **12**

**5 Why this should help and what it achieves** **12**

**6 The same algorithm as pseudocode** **12**

**7 Glossary** **13**

# 1 The whole method in one page

**For a school student.** We want a computer to learn to recognise animals from photos without anyone telling it what is in the photos. The problem: in our photos cats almost always sit on sofas and dogs almost always run on grass. A lazy learner decides "sofa means cat" and fails on a cat in the garden. So we bring in a helper who already knows many words, a big pretrained model called CLIP. Before the lessons start, the helper goes through every photo and colours it in like a colouring book: "these squares are cat, these are sofa, these are window". During the lessons the learner looks at pieces of the photos and, for every small square of a piece, has to say which words belong there. Because the cat and the sofa sit in *different* squares, the learner must learn what a cat looks like by itself and what a sofa looks like by itself. To stop the learner from simply memorising the answer sheet of each photo, we check its answers with a rule built from *other* photos.

**For the supervisor.** SCD adds one loss to SimCLR. Offline, a frozen CLIP ViT-B/32 with ClearCLIP's last block computes, for every training image, a $14 \times 14$ map of soft assignments to $K$ concept factors (text directions of synonym groups found by SpLiCE). Online, every SimCLR view's crop box and flip cut the matching part out of its image's map and resample it to $7 \times 7$. The $7 \times 7 \times 512$ `layer4` output of the ResNet-18 student must predict these $K$ values at each of its 49 locations through a ridge regression that is fitted in closed form on one half of the batch and evaluated on the other half. The held-out squared error, weighted by $\lambda = 5$ after a warm-up, is added to the SimCLR loss.

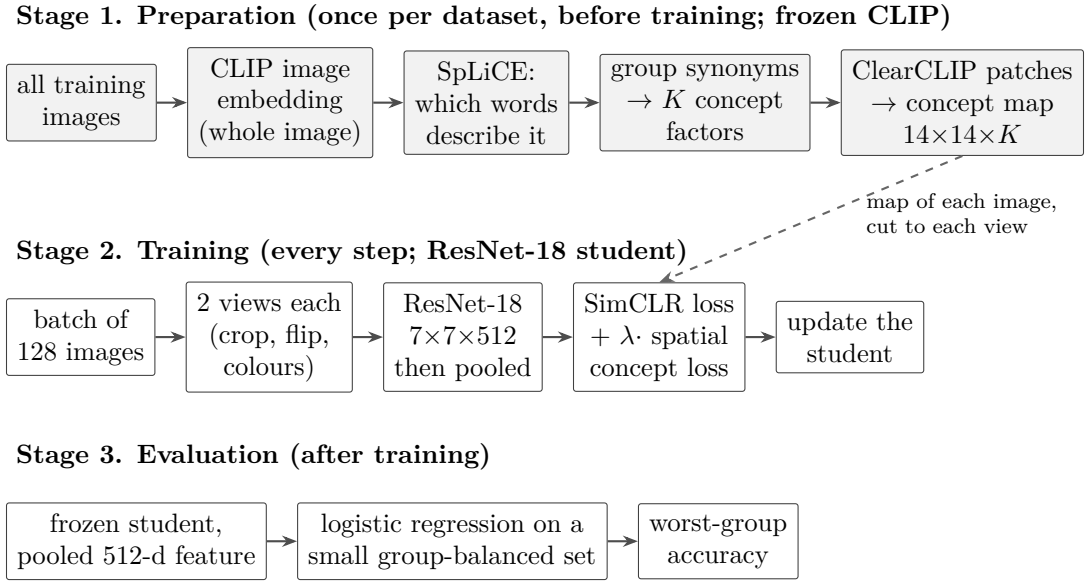



**Figure 1:** The three stages. CLIP appears only in stage 1. After stage 1 only the maps are kept; after stage 2 only the ResNet-18 is kept; the maps, the projection head and every ridge fit are thrown away.

One correction to a natural first guess: SpLiCE runs on the embedding of the *whole* image from ordinary CLIP and its job is to choose the vocabulary. The maps come from ClearCLIP's *patch* embeddings, which are compared with the text directions of that vocabulary. SpLiCE is never applied to patches.

**Table 1:** Shapes at every step for MetaShift ($N = 1{,}700$ images, $K = 130$ factors, batch $B = 128$).

| Step | Object | Shape |
|---|---|---|
| 1.1 | CLIP image embedding | $N \times 512$ |
| 1.2 | SpLiCE codes | $N \times 10{,}000$, about 9 non-zero per image |
| 1.3 | factor text directions $d_F$ | $K \times 512$ |
| 1.4 | image for the maps | $448 \times 448$ pixels, 9 windows of $224 \times 224$ |
| 1.5 | ClearCLIP patch embeddings | per window $7 \times 7 \times 512$ |
| 1.6 | stored concept maps | $N \times K \times 14 \times 14$ in float16 (87 MB for MetaShift) |
| 2.1 | views of a batch | $2B = 256$ images of $224 \times 224$, each with a crop box and a flip flag |
| 2.2 | student feature map (`layer4`) | $256 \times 512 \times 7 \times 7$ |
| 2.2 | pooled feature, projection | $256 \times 512$, then $256 \times 128$ |
| 2.3 | warped targets | $256 \times K \times 7 \times 7$ |
| 2.4 | rows of one half | $64 \cdot 2 \cdot 49 = 6{,}272$ rows of 512 features and $K$ targets |
| 2.4 | ridge solution $W_A$ | $512 \times K$ |

# 2 Stage 1: preparation, once per dataset

Stage 1 runs before training starts, on the training images only, with a frozen CLIP model (OpenCLIP ViT-B/32, LAION-2B weights; Radford et al., 2021). Nothing in it is trained and no label is used. Its output is one file: a concept map for every training image.

## 2.1 Step 1.1: one vector per image from CLIP

**In plain words.** CLIP was trained on hundreds of millions of photos with captions, so it learned to place a photo and a sentence that describes it close together. Give it a photo and it returns a list of 512 numbers, a "fingerprint" that sits close to the fingerprints of words describing the photo.

**Precisely.** The image is resized to 224 pixels and passed through CLIP's vision transformer. The class token of the last layer, projected into the joint space, is the embedding $z_i \in \mathbb{R}^{512}$. Every word $k$ of a vocabulary has a text embedding $t_k \in \mathbb{R}^{512}$ from CLIP's text encoder.

**Why this way.** CLIP knows thousands of visual concepts from external data. It is the only source of "names" in the method; the student never sees CLIP itself.

## 2.2 Step 1.2: SpLiCE turns the vector into a few words

**In plain words.** The 512 numbers are hard to read. SpLiCE rewrites the fingerprint as a short recipe: "0.41 cat + 0.30 sofa + 0.12 blanket + ...", using as few words as possible. About nine words are enough per image.

**Precisely.** With a fixed mean $\mu$ and the dictionary $D$ of text directions (10,000 LAION words for MetaShift), SpLiCE solves (Bhalla et al., 2024)

$$u_i = \frac{z_i - \mu}{\|z_i - \mu\|}, \qquad c_i = \arg\min_{c \geq 0} \|u_i - D^\top c\|_2^2 + 0.25\,\|c\|_1. \tag{1}$$

$c_i$ has one entry per word and about nine of them are non-zero.

**Why this way.** We need to know *which* concepts occur in this dataset and how often, without labels. Running SpLiCE over all 1,700 images gives exactly that list. The rest of the method only uses this list; the per-image recipe $c_i$ is used for counting frequencies and for the grouping in the next step.

### 2.3 Step 1.3: group synonyms into concept factors

**In plain words.** The dictionary has *cat*, *kitten* and *kitty* as separate words. For our purpose they are one thing. We merge words that mean the same into one "concept factor". A factor is a small bag of synonyms with one common direction.

**Precisely.** Keep words active on 0.2% to 95% of the images. Two words are similar when the cosine of their raw text embeddings is high; the pair is vetoed when the images' responses $u_i^\top d_k$ to the two words correlate below 0.5. Average-linkage clustering cut at similarity 0.85 forms the groups. A factor $F$ gets the text direction

$$d_F = \operatorname{norm}\Big(\tfrac{1}{|F|}\textstyle\sum_{k\in F}\operatorname{norm}(t_k)\Big) \tag{2}$$

and factors active on 2% to 90% of the images are kept: $K = 130$ on MetaShift, 242 on Waterbirds, 147 on Spur-CIFAR10. Examples: {kitty, kitten, cat}, {canine, dog}, {couch, sofa}.

**Why this way.** Synonyms would otherwise compete with each other in every square of the map (step 1.6) and split the mass of one thing over several names. The thresholds and the reason why the text cosine works and co-activation fails are in Section 4.4 of the main report.

### 2.4 Step 1.4: why plain CLIP is not enough for maps

**In plain words.** We do not only want to know *that* the photo contains a cat; we want to know *where*. CLIP's vision transformer cuts the image into small squares (patches) and keeps one vector per square. One would hope that the vector of a square on the cat says "cat". In ordinary CLIP it does not: in the very last layer every square "listens" to the same few squares that collect the summary of the whole picture, so every square ends up repeating the summary. Asking a square on the sofa what it shows gives "cat, sofa, room", the same answer as every other square.

**Precisely.** CLIP is trained to align only the class token with text; nothing asks individual patch tokens to match text. In the last block the query–key attention of every patch concentrates on the same few tokens (Wang et al., 2024; Zhou et al., 2022); large ViTs store global information in a few high-norm tokens (Darcet et al., 2024); the residual branch then adds this global signal to every patch. ClearCLIP (Lan et al., 2024) changes only the last block: (i) query–query attention in place of query–key, so a patch attends to patches that look like itself; (ii) no residual connection and (iii) no feed-forward branch. With $X$ the normalized tokens entering the last block,

$$P = \operatorname{norm}\Big(\operatorname{LN}\big(\operatorname{softmax}(QQ^\top/\sqrt{d_h})\,VW_O\big)W_{\text{proj}}\Big), \qquad Q = XW_Q,\ V = XW_V, \tag{3}$$

gives one unit-length vector $p(u) \in \mathbb{R}^{512}$ per patch $u$ in the same space as the text directions $d_F$.

**Why this way.** ClearCLIP needs no training and no labels, so the restriction of the thesis holds. MaskCLIP and SCLIP follow the same idea; ClearCLIP reports the cleanest maps of the three. The code is `patch_embeddings` in `cospro/pipeline/concept_maps.py`.

## 2.5 Step 1.5: reading the image through windows

**In plain words.** ViT-B/32 cuts a 224-pixel image into a $7 \times 7$ grid of squares that are 32 pixels wide. That is coarse: a cat's head and the cushion next to it fall into one square. So we enlarge the photo to 448 pixels and look at it through nine overlapping 224-pixel windows, like reading a large map through a magnifying glass moved in steps of half its width. Each window gives its own $7 \times 7$ answers and where windows overlap we average them.

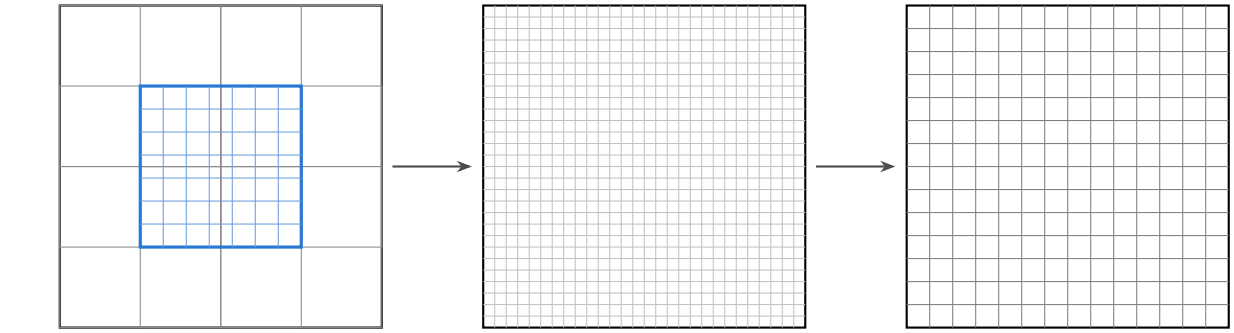



**Figure 2:** From windows to the $14 \times 14$ grid. The windows start at pixels 0, 112 and 224 in each direction, so a cell is covered by one, two or four windows.

**Precisely.** The image is resized (both sides, bicubic) to $448 \times 448$. Windows start at offsets $\{0, 112, 224\}$ in each direction: $3 \times 3 = 9$ windows. Each window passes through ClearCLIP and gives $7 \times 7$ patch vectors. The cosine of every patch with every factor direction, $s_F(u) = p(u)^\top d_F$, is spread onto a grid of 16-pixel cells ($448/16 = 28$), averaged where windows overlap and average-pooled to $14 \times 14$ (`assemble_windows`).

**Why this way.** Each window stays at 224 pixels, the resolution CLIP was trained on, while the effective grid becomes 16 pixels. The stride of 112 makes every window boundary fall on a whole cell. SCLIP and ClearCLIP evaluate dense predictions with the same sliding-window scheme (Lan et al., 2024; Wang et al., 2024).

## 2.6 Step 1.6: from scores to a concept map

**In plain words.** For every one of the $14 \times 14 = 196$ squares we now have 130 scores: how much the square looks like "cat", like "sofa", like "window" and so on. We turn the scores of each square into shares that add up to one, so that each square says "I am 80% cat, 15% blanket, 5% the rest". Picture a colouring book in which every square is coloured with the word that fits it best (Figure 3).

**Precisely.** At every cell $u$ of image $i$,

$$M_{iF}(u) = \frac{\exp\left(s_{iF}(u)/T\right)}{\sum_{G=1}^{K} \exp\left(s_{iG}(u)/T\right)}, \qquad T = 0.01. \tag{4}$$

The map $M_i \in [0,1]^{K \times 14 \times 14}$ is stored in float16. When training loads the maps, every factor is standardized over all images and cells, $\tilde{M}_{iF}(u) = (M_{iF}(u) - m_F)/s_F$, so every factor has mean 0 and variance 1.

*Illustrative example.* A cell on the cat has cosines 0.27 with *cat*, 0.25 with *blanket* and 0.21 with *sofa*. Divided by $T = 0.01$ these become 27, 25 and 21; after the softmax the cell is about 88% cat, 12% blanket and 0.2% sofa. With $T = 1$ the three would be almost equal (34%, 33%, 32%).

**Why this way.** $T = 0.01$ is CLIP's own scale: CLIP multiplies its cosines by a learned logit scale of 100 before its softmax (Radford et al., 2021). Cosines of a patch with different words differ by a few hundredths, so this scale is what makes one word win per cell. Standardization stops frequent factors (blanket, lawn) from dominating the squared error in step 2.4. Why $14 \times 14$ and not $28 \times 28$ or $7 \times 7$: step 2.3 cuts a crop out of this map; the smallest SimCLR crop spans about $6 \times 6$ cells of a $14 \times 14$ map, enough to fill the student's $7 \times 7$ grid and the stored maps stay small (87 MB for MetaShift, 455 MB for Waterbirds).

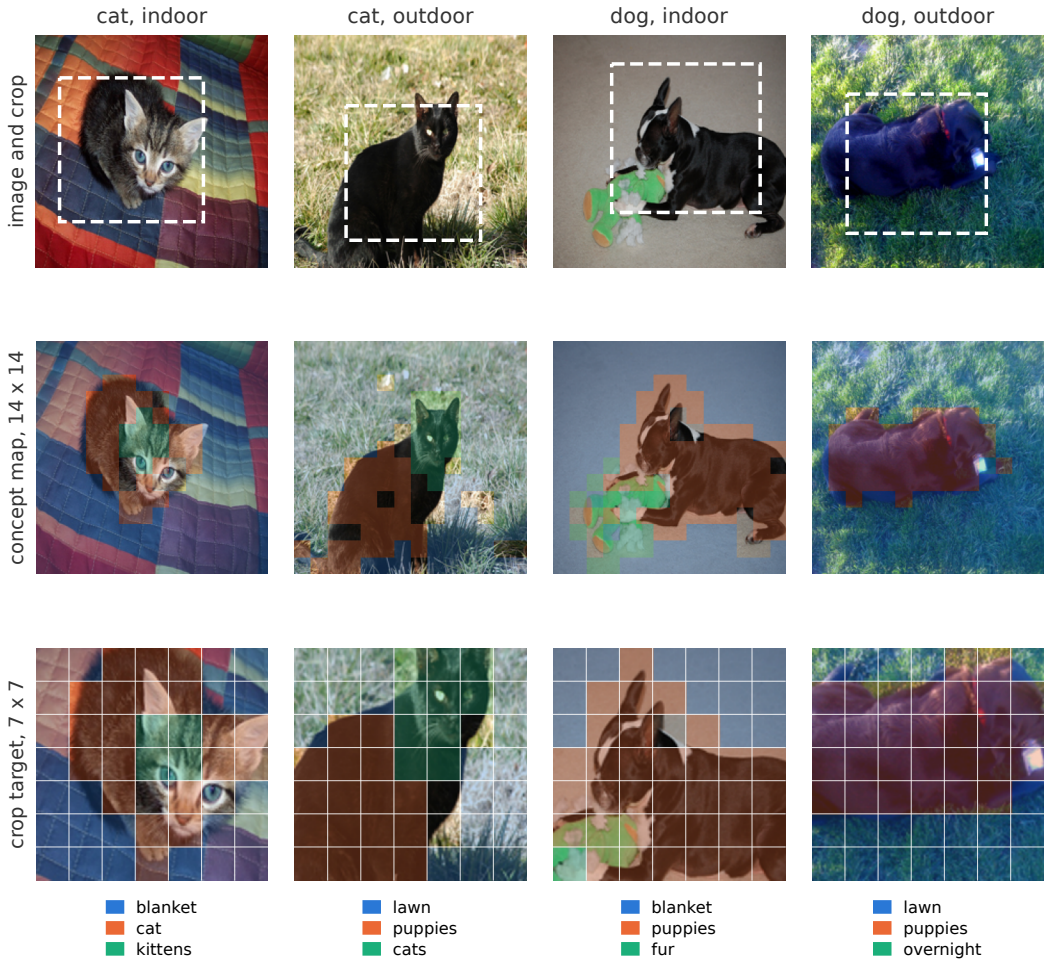



**Figure 3:** Real MetaShift maps. Top: image with one SimCLR crop (dashed). Middle: the winning factor of each of the $14 \times 14$ cells. Bottom: the $7 \times 7$ targets that this crop receives (step 2.3). The teacher makes mistakes: part of the black cat in the second image is labelled "puppies".

**Output of stage 1.** One tensor of shape $N \times K \times 14 \times 14$: for every training image and every square of it, the share of each concept factor. Building it takes about 40 seconds on one GPU for MetaShift (`scripts/build_concept_maps.sbatch`). From here on CLIP is never used again.

## 3 Stage 2: one training step

Stage 2 is ordinary SimCLR training of a ResNet-18 from random weights for 500 epochs, with one extra loss. One epoch passes over all 1,700 images in batches of 128. Here is what happens in one step.

### 3.1 Step 2.1: two views of every image

**In plain words.** We take 128 photos. From each photo we make two "versions" by random edits: cut out a random piece and blow it up to full size (crop), maybe mirror it left to right (flip), change brightness and colours, sometimes make it grey. The two versions of one photo look different but show the same thing.

**Precisely.** The SSL transform of every view applies, in this order:

- **Crop** (`RandomResizedCrop`): choose a rectangle covering 20% to 100% of the image area with aspect ratio between 3:4 and 4:3, at a random position; resize it to $224 \times 224$.
- **Flip** (`RandomHorizontalFlip`): mirror left–right with probability 0.5.
- **Colour**: colour jitter (brightness, contrast, saturation 0.4, hue 0.1) with probability 0.8; greyscale with probability 0.2.

The transform *records* each view's crop box as fractions of the image (top, left, height, width) and its flip flag (`cospro/data/transforms.py`). The batch thus holds $2 \times 128 = 256$ views and 256 records.

**Why this way.** SimCLR learns by recognising that two edited versions come from the same photo (Chen et al., 2020). Crop and flip are the only edits that move pixels; the colour edits keep every pixel in place. That is why only the crop box and the flip flag are recorded: they tell us which part of the concept map each view shows.

### 3.2 Step 2.2: the student looks at each view

**In plain words.** The student network looks at each $224 \times 224$ view and, step by step, shrinks it to a $7 \times 7$ grid. Each of the 49 grid cells holds 512 numbers that describe what the student sees in that part of the view. Then the student averages the 49 cells into one summary of 512 numbers for the whole view.

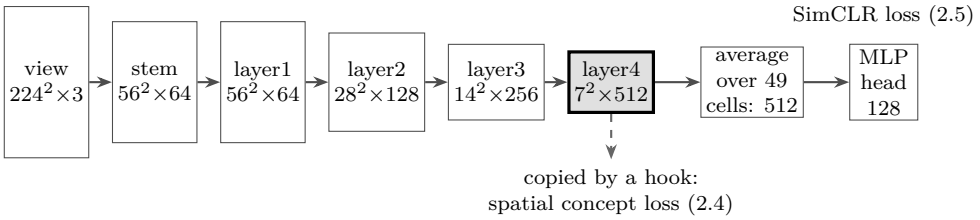



**Figure 4:** The ResNet-18 student. The resolution halves five times: $224/2^5 = 7$.

**Precisely.** The ResNet-18 (He et al., 2016) maps a view to $H \in \mathbb{R}^{512 \times 7 \times 7}$ at the output of `layer4`. Global average pooling gives $h = \frac{1}{49} \sum_u H(u) \in \mathbb{R}^{512}$, the feature the probe will read after training. An MLP head maps $h$ to $g \in \mathbb{R}^{128}$ for the SimCLR loss. A forward hook keeps a copy of $H$ for the spatial loss.

**Why this way.** Location $(r, c)$ of $H$ sits over the 32-pixel square $(r, c)$ of the view, so each of the 49 vectors can be asked about its own part of the view. After averaging, a cat and a sofa in the same view become one vector; before averaging they are still at different locations. Class activation maps rest on the same property of this last map (Zhou et al., 2016).

**Two different "$7 \times 7$" grids.** CLIP's window in step 1.5 also has $7 \times 7$ patches, because $224/32 = 7$. The student's $7 \times 7$ in this step has the same size for a different reason ($224/2^5 = 7$). The two grids

never meet directly: CLIP's grid is turned into the $14 \times 14$ map of the whole image and step 2.3 resamples that map to the student's grid of each view.

### 3.3 Step 2.3: cut the matching piece out of the concept map

**In plain words.** Each view shows only a piece of the photo, maybe mirrored. Its "answer sheet" must show the same piece. So we take the photo's colouring-book map, cut out the same rectangle that the crop cut out of the photo, shrink it to $7 \times 7$ squares and mirror it if the view was mirrored. Now square $(r, c)$ of the answer sheet describes exactly the part of the view that the student's cell $(r, c)$ looks at.

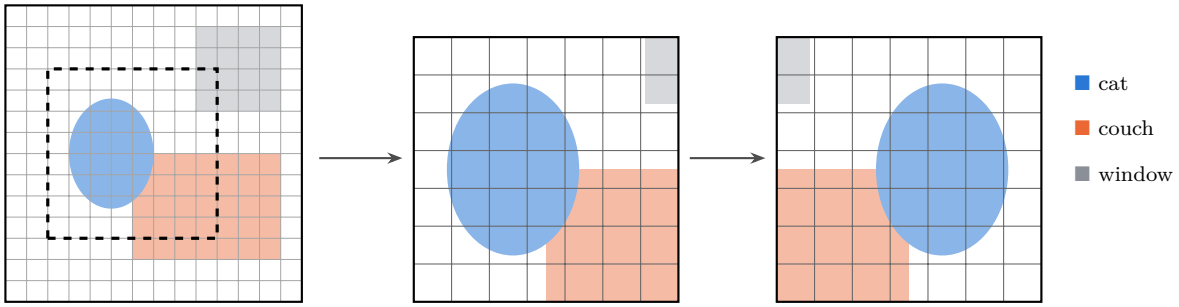



**Figure 5:** Warping the map onto one view (toy scene). Each of the $K$ channels of the map is cut and resampled in the same way; the figure colours each cell by its winning factor.

**Precisely.** For a view with box (top $a$, left $b$, height $\eta$, width $\omega$, all as fractions of the image), the box in map coordinates is $[14b,\ 14(b+\omega)] \times [14a,\ 14(a+\eta)]$. RoIAlign (He et al., 2017) divides it into $7 \times 7$ bins and averages $2 \times 2$ bilinear samples of $\tilde{M}_i$ inside each bin; a flipped view gets its target mirrored along the width (`warp_maps`). The result is $Y^{(v)} \in \mathbb{R}^{K \times 7 \times 7}$, the target of view $v$.

**Why this way.** The crop box is stored as fractions of the original image and the map covers the whole image, so the box can be placed on the map whatever the original size; resizing the image to $448 \times 448$ for the map does not change the fractions. RoIAlign reads the map at exact fractional positions, so a crop that starts in the middle of a cell gets the right values. A crop of 20% area spans about $0.45 \times 14 \approx 6$ cells per side, so even the smallest crop has real content for its $7 \times 7$ target.

### 3.4 Step 2.4: the spatial concept loss

This step is where the concepts enter training. It has four parts.

#### (a) Make 6,272 question–answer pairs per half.

**In plain words.** Every one of the 49 cells of every view becomes one question: "here are your 512 numbers for this cell; which concepts are here?". The answer is the matching cell of the answer sheet from step 2.3.

**Precisely.** Each location $u$ of view $v$ gives a row $r = H^{(v)}(u)/\|H^{(v)}(u)\| \in \mathbb{R}^{512}$ and a target $y = Y^{(v)}(u) \in \mathbb{R}^K$. The 128 images of the batch are split into halves $A$ and $B$ of 64 images, both views of an image always in the same half. Each half holds $64 \times 2 \times 49 = 6{,}272$ pairs $(r, y)$.

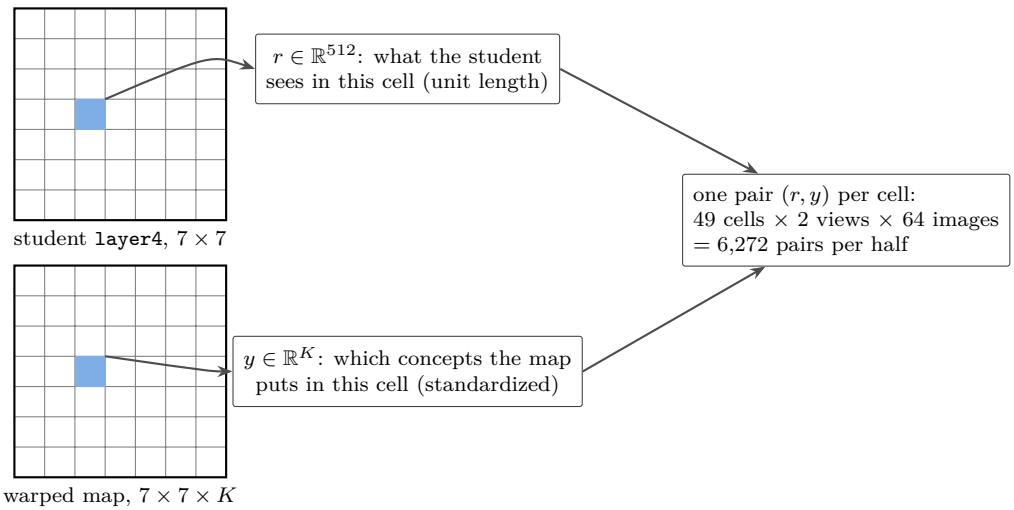



**Figure 6:** The same cell of the student's feature map and of the warped map form one question–answer pair.

### (b) Learn a translation rule on half $A$.

**In plain words.** On half $A$ we find the best simple rule that turns the student's 512 numbers into concept values. The rule is a table $W$: for each concept, a direction in the student's feature space; the predicted value of "cat" in a cell is how far the cell's vector points along the cat direction. The rule is found by a formula in one go, with no training.

**Precisely.** Weighted ridge regression, solved in closed form,

$$W_A = \left(\bar{R}_A^\top \Omega_A \bar{R}_A + \rho I\right)^{-1} \bar{R}_A^\top \Omega_A \left(Y_A - \bar{y}_A\right) \ \in \mathbb{R}^{512 \times K}, \tag{5}$$

where $\bar{R}_A$ stacks the 6,272 rows of $A$ minus their weighted mean $\bar{r}_A$, $Y_A$ the targets, $\bar{y}_A$ their weighted mean, $\Omega_A$ the per-row weights and $\rho = 1$. The solve is one $512 \times 512$ linear system in float32.

### (c) Test the rule on half $B$.

**In plain words.** We use the rule from half $A$ to predict the concepts in every cell of half $B$ and measure how wrong it is. Then we swap: a rule from $B$ is tested on $A$. The loss is the average mistake on the half the rule has not seen.

**Precisely.**

$$\hat{Y}_B = \left(R_B - \bar{r}_A\right) W_A + \bar{y}_A, \qquad \mathcal{L}_{\text{spatial}} = \tfrac{1}{2}\Big(\operatorname{err}(\hat{Y}_B, Y_B) + \operatorname{err}(\hat{Y}_A, Y_A)\Big), \tag{6}$$

with err the weighted mean over views of $\frac{1}{49K}\|\hat{Y}_v - Y_v\|^2$ (`spatial_cross_fit_loss`). Because the targets have variance 1, $1 - \text{err}$ is the held-out explained variance, logged during training.

### (d) Push the student.

**In plain words.** The rule itself is thrown away after the step. What we keep is the lesson for the student: "change your 512 numbers so that next time a rule learned on some photos works better on other photos".

**Precisely.** Gradients of $\mathcal{L}_{\text{spatial}}$ flow through the prediction and through the solve for $W_A$ into $H$ and the ResNet weights (Bertinetto et al., 2019). The ridge has no parameters of its own; it is solved again in every step from the current features.

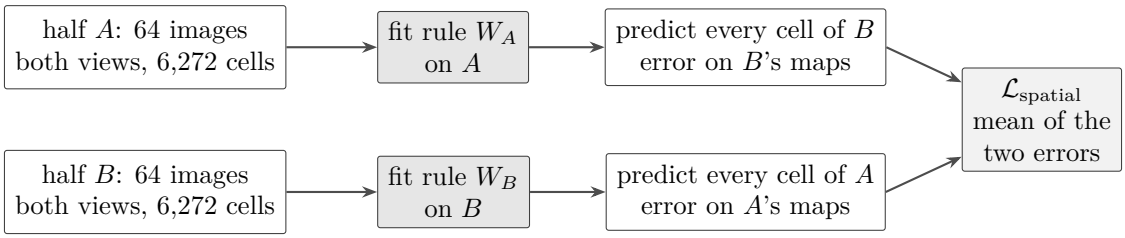



**Figure 7:** Cross-fitting: each rule is tested only on images it was not fitted on.

**Why this way.** The cross-fit is the heart of the method and the lesson of the image-level experiments. Suppose the student learned to recognise each photo individually ("this is photo 371") and stored the answers of photo 371 somewhere in its features. A rule fitted on half $A$ could then reproduce $A$'s answers, but it would be useless on half $B$, whose photos it has never seen; memorising photos earns nothing. The only way to lower the loss is a *shared* code: cat-ness in the same direction in every photo, couch-ness in another direction, so that one rule works everywhere. This is the sample splitting of double machine learning (Chernozhukov et al., 2018) applied to a training loss. With a trained head in its place, the student fitted shuffled targets almost as well as real ones on MetaShift (Zhang et al., 2017).

The other choices of this step:

- **Unit-length rows and an absolute ridge $\rho = 1$.** If the student could shrink its features, it would weaken the ridge and let the rule overfit. Unit rows fix the scale; $\rho = 1$ keeps the $512 \times 512$ system well conditioned (Hoerl and Kennard, 1970). An earlier version without them produced NaN features.
- **A linear rule.** The probe that evaluates the student after training is linear, so the concepts must be *linearly* readable from the features.
- **Both views in one half.** Otherwise half $B$ would contain the other view of a photo in half $A$, and recognising the photo would pay off again.
- **6,272 rows against 512 dimensions.** Twelve times more rows than unknowns per concept: the fit is well posed. A batch of 128 gives this ratio; the image-level version had only 128 rows per half.

**The 20% hold-out.** A fixed fifth of the training images (340 on MetaShift) gets weight $\omega = 0$ in $\Omega$: their cells neither shape any rule nor count in the error. SimCLR still trains on them. After training they show whether the student's concepts carry over to images that never entered the concept loss. The arm with this hold-out also scored higher than the arm without it, so it stayed in the main configuration.

### 3.5 Step 2.5: the SimCLR loss, unchanged

**In plain words.** Among the 256 views in the batch, every view has exactly one partner: the other version of the same photo. The student must make the summaries of partners similar and the summaries of all other views different. It is a game of "find your twin among 255 strangers".

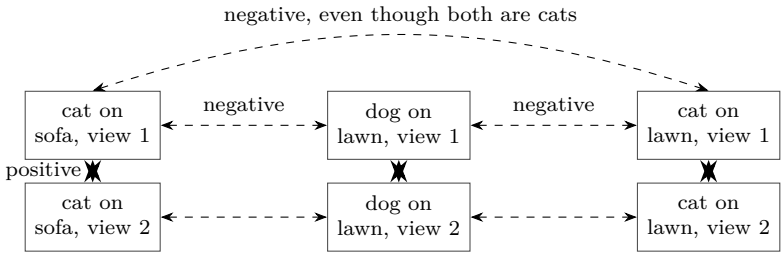



**Figure 8:** SimCLR pairs. Each view's positive is the other view of its own image; all 254 other views are negatives, including other cats.

**Precisely.** With $g_v \in \mathbb{R}^{128}$ the normalized projection of view $v$, $v^+$ its partner and $\tau = 0.05$, the NT-Xent (InfoNCE) loss (Chen et al., 2020; van den Oord et al., 2018) is

$$\mathcal{L}_{\text{SimCLR}} = -\frac{1}{2B} \sum_{v=1}^{2B} \log \frac{\exp(g_v^\top g_{v^+}/\tau)}{\sum_{w \neq v} \exp(g_v^\top g_w/\tau)}. \tag{7}$$

**Why this way.** SimCLR is the self-supervised method all our comparisons share. It needs no labels, but it rewards any feature that tells photos apart, the sofa as much as the cat; on a dataset where sofa and cat come together, it has no reason to separate them. The spatial loss of step 2.4 adds that reason.

### 3.6 Step 2.6: add the two losses and update

**In plain words.** The final score is the SimCLR score plus the concept score times a weight. For the first ten epochs the weight is zero, so the student first learns ordinary SimCLR; over the next ten epochs the weight grows to 5 and stays there.

**Precisely.**

$$\mathcal{L} = \mathcal{L}_{\text{SimCLR}} + \lambda(t)\, \mathcal{L}_{\text{spatial}}, \qquad \lambda(t) = \begin{cases} 0, & t \leq 10, \\ 5\,(t-10)/10, & 10 < t < 20, \\ 5, & t \geq 20, \end{cases} \tag{8}$$

with $t$ the epoch. SGD (learning rate 0.01, decayed by 0.1 at epochs 350, 400 and 450, weight decay $10^{-4}$) updates the ResNet and the MLP head; float16 autocast runs the network, float32 the ridge solve.

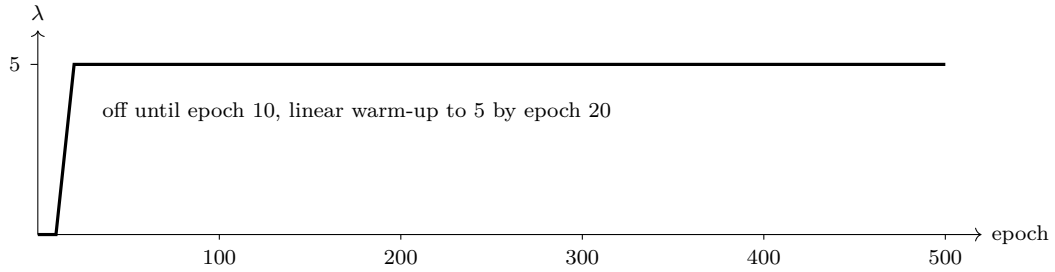



**Figure 9:** Weight of the spatial concept loss over training.

**Why this way.** SimCLR first forms usable features; a concept loss on random features at epoch 0 would mostly add noise. The weight 5 came from a grid of 1, 3, 5 and 10; weight 10 added nothing.

## 4 Stage 3: evaluation

**In plain words.** After training we keep only the student. For each photo it gives one summary of 512 numbers. A very simple classifier (a line through the space of summaries) learns cat versus dog from a small set with equally many photos of each kind: cats indoors, cats outdoors, dogs indoors, dogs outdoors. We then check its accuracy on each of the four kinds separately and report the worst one.

**Precisely.** Every 25 epochs a logistic regression (L2 coefficient 0.001) is fitted on the pooled 512-d features of a group-balanced subset of the training split (400 images on MetaShift) and evaluated on the validation split; the reported value is the mean of the last four probes (epochs 425 to 500). Labels appear only here.

**Why this way.** A balanced linear probe recovers the class whenever the representation holds the class and the spurious attribute in separate directions (Kirichenko et al., 2023). The spatial loss aims for exactly that: a cat direction that the probe can use on cats outdoors.

## 5 Why this should help and what it achieves

**The argument.** At the level of whole photos, "cat" and "sofa" arrive together on 88% of MetaShift's training images; a student that only sees whole photos can satisfy every whole-photo target with one fused "cat-and-sofa" feature. At the level of cells, the cat and the sofa sit in different cells of *every* photo. A single rule that predicts "cat here, sofa there" for photos it has not seen requires the student to code cat-ness and sofa-ness separately, in the same way across photos. The balanced probe can then pick the cat direction.

**What it achieves.** Validation WGA rises over SimCLR by 6.1 points on Waterbirds (46.7 to 52.8), 6.8 on MetaShift (44.3 to 51.1) and 22.5 on Spur-CIFAR10 (15.6 to 38.1). The same loss with another photo's maps (shuffled control) stays at the SimCLR level, so the gain comes from each photo's own maps. On the 20% held-out images the student predicts the concepts as well as on the images inside the loss: it learned shared concepts.

**Where it stops.** A cell of the student's $7 \times 7$ map is computed from a large part of the view (Luo et al., 2016), so the cell over the cat also sees the sofa. The cheapest way to predict "cat here" still uses the sofa, which is right 88% of the time. The audit confirms it: the student's cat factor scores 0.47 on unseen cats indoors and 0.07 on unseen cats outdoors, against 0.92 and 0.88 for CLIP. SCD makes the cat feature stronger; it does not yet free it from the sofa. Section 8 of the main report describes the attempts to go further.

## 6 The same algorithm as pseudocode

```
# ---------- Stage 1: once per dataset (frozen CLIP, no labels) ----------
z      = clip.encode_image(images_224)               # N x 512
codes  = splice(z, vocabulary, l1=0.25)              # N x 10000, ~9 non-zero
factors, d = meaning_grouping(codes, text_emb)       # K factors, d: K x 512
for image in dataset:
    big   = resize(image, 448, 448)
    score = zeros(K, 28, 28); count = zeros(28, 28)
    for top in (0, 112, 224):
```

```
        for left in (0, 112, 224):
            p = clearclip_patches(big[top:top+224, left:left+224])  # 7x7x512
            s = p @ d.T                                              # 7x7xK
            score += s spread to 2x2 cells; count += 1
    maps[image] = softmax(avgpool(score / count, 14) / 0.01, dim=K)  # K x14x14
maps = standardize_per_factor(maps)

# ---------- Stage 2: every training step ----------
for images in batches_of_128:
    v1, box1 = augment(images); v2, box2 = augment(images)  # crop+flip recorded
    H = resnet.layer4(cat(v1, v2))           # 256 x 512 x 7 x 7  (hook)
    g = mlp(avgpool(H))                       # 256 x 128
    loss_ssl = nt_xent(g, temperature=0.05)
    Y = roi_align(maps[images], cat(box1, box2), 7x7)  # then flip flipped views
    R = normalize(H per location)             # rows of 512
    A, B = halves of the 128 images (both views together)
    W_A = ridge(R[A], Y[A], weights, rho=1); W_B = ridge(R[B], Y[B], ...)
    loss_spatial = (mse(R[B] @ W_A, Y[B]) + mse(R[A] @ W_B, Y[A])) / 2
    loss = loss_ssl + lambda(epoch) * loss_spatial
    loss.backward(); sgd.step()   # grads pass through the solve

# ---------- Stage 3: evaluation ----------
probe = logistic_regression(avgpool(resnet(balanced_images)), labels)
report worst_group_accuracy(probe, validation)
```

# 7 Glossary

| Term | Meaning |
|---|---|
| View | One randomly edited copy of a photo (crop, flip, colour); SimCLR uses two per photo |
| Crop box | The rectangle a view was cut from, stored as fractions of the photo |
| Flip | Left–right mirroring, applied to half of the views |
| Positive / negative | In SimCLR: the other view of the same photo / every view of another photo |
| Patch | A $32 \times 32$-pixel square that CLIP's vision transformer turns into one vector |
| ClearCLIP | CLIP with a modified last block whose patch vectors match text locally |
| Concept factor | A group of synonyms ({cat, kitten, kitty}) with one text direction |
| Concept map | For one photo: $14 \times 14$ cells, each with shares of the $K$ factors summing to 1 |
| Warping | Cutting a view's crop box out of the map and resampling it to $7 \times 7$ (RoIAlign) |
| Feature map | The student's `layer4` output, $7 \times 7$ cells of 512 numbers |
| Cross-fit | Fit the rule on one half of the batch, measure its error on the other half |
| Ridge | Least-squares fit of a linear rule with a penalty $\rho$ that keeps it stable |
| Hold-out | 20% of photos that never enter the concept loss, used to check generalization |
| WGA | Worst-group accuracy: accuracy on the hardest of the four (class, background) groups |

# References


L. Bertinetto, J. F. Henriques, P. H. S. Torr and A. Vedaldi. Meta-learning with Differentiable Closed-Form Solvers. *ICLR*, 2019. `https://arxiv.org/abs/1805.08136`.

U. Bhalla, A. Oesterling, S. Srinivas, F. P. Calmon and H. Lakkaraju. Interpreting CLIP with Sparse Linear Concept Embeddings (SpLiCE). *NeurIPS*, 2024. `https://arxiv.org/abs/2402.10376`.

T. Chen, S. Kornblith, M. Norouzi and G. Hinton. A Simple Framework for Contrastive Learning of Visual Representations. *ICML*, 2020. `https://arxiv.org/abs/2002.05709`.

V. Chernozhukov et al. Double/Debiased Machine Learning for Treatment and Structural Parameters. *The Econometrics Journal*, 21(1), 2018. https://arxiv.org/abs/1608.00060.

T. Darcet, M. Oquab, J. Mairal and P. Bojanowski. Vision Transformers Need Registers. *ICLR*, 2024. https://arxiv.org/abs/2309.16588.

K. He, X. Zhang, S. Ren and J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016. https://arxiv.org/abs/1512.03385.

K. He, G. Gkioxari, P. Dollár and R. Girshick. Mask R-CNN. *ICCV*, 2017. https://arxiv.org/abs/1703.06870.

A. E. Hoerl and R. W. Kennard. Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 1970.

P. Kirichenko, P. Izmailov and A. G. Wilson. Last Layer Re-Training is Sufficient for Robustness to Spurious Correlations. *ICLR*, 2023. https://arxiv.org/abs/2204.02937.

M. Lan et al. ClearCLIP: Decomposing CLIP Representations for Dense Vision-Language Inference. *ECCV*, 2024. https://arxiv.org/abs/2407.12442.

W. Luo, Y. Li, R. Urtasun and R. Zemel. Understanding the Effective Receptive Field in Deep Convolutional Neural Networks. *NeurIPS*, 2016. https://arxiv.org/abs/1701.04128.

A. van den Oord, Y. Li and O. Vinyals. Representation Learning with Contrastive Predictive Coding. *arXiv:1807.03748*, 2018. https://arxiv.org/abs/1807.03748.

A. Radford et al. Learning Transferable Visual Models From Natural Language Supervision. *ICML*, 2021. https://arxiv.org/abs/2103.00020.

F. Wang, J. Mei and A. Yuille. SCLIP: Rethinking Self-Attention for Dense Vision-Language Inference. *ECCV*, 2024. https://arxiv.org/abs/2312.01597.

C. Zhang, S. Bengio, M. Hardt, B. Recht and O. Vinyals. Understanding Deep Learning Requires Rethinking Generalization. *ICLR*, 2017. https://arxiv.org/abs/1611.03530.

B. Zhou, A. Khosla, A. Lapedriza, A. Oliva and A. Torralba. Learning Deep Features for Discriminative Localization. *CVPR*, 2016. https://arxiv.org/abs/1512.04150.

C. Zhou, C. C. Loy and B. Dai. Extract Free Dense Labels from CLIP. *ECCV*, 2022. https://arxiv.org/abs/2112.01071.